

线段树入门级总结

By Jsn1993

最近学习了线段树这种数据结构，也做了不少关于线段树的入门级题目，大致对线段树有了一个了解。稍稍总结一下（文字没有参考别人的成分）。

一、初识线段树

1、称谓及意义

线段树我习惯用英文称呼它（Interval Tree）。Interval 代表区间，恰恰说明了线段树是一种基于区间的数据结构。

2、线段树的表示

线段树是一棵二叉树，它的左右儿子也都是一棵线段树。

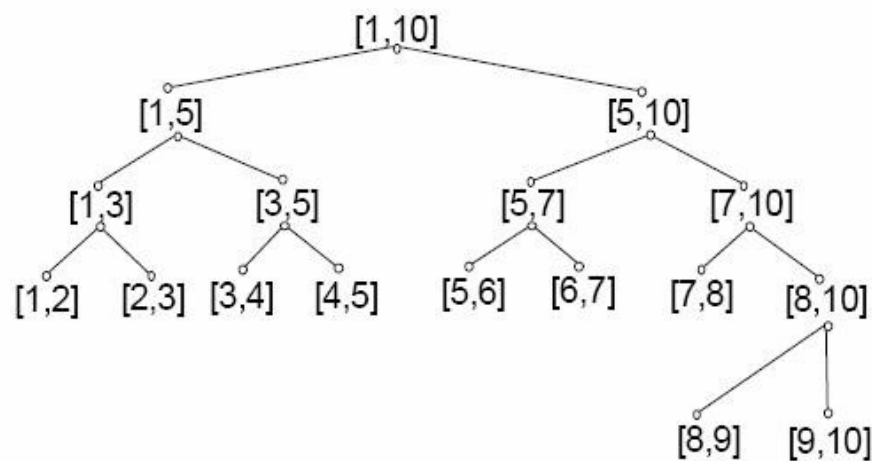
在最坏情况下节点的标号最大为 $N \text{ shl } 2$ （图 1）。

线段树的每个节点代表一个区间 $[L,R]$ ，其中，假设根代表一个区间 $[L_0,R_0]$ 。

取 $\text{Mid}=(L_0+R_0) \text{ shr } 1$ ，则根的左儿子代表的区间为 $[L_0,\text{mid}]$ ，右儿子为 $[\text{Mid}+1,R_0]$ 。

线段树的每个叶子节点代表的区间 $[L,R]$ ，其中 $L=R$ 。

具体可参考图 1（Quote From Rujia Liu's ppt）：



图一

3、线段树区间的意义

在实际应用中， $[L,R]$ 往往只代表这个区间的左右界，并不是代表具体的值。

往往在需要对某个区间进行操作并且进行了操作后，由于区间的包含性与传递性，需要更新这个区间的父亲与儿子。

二、 线段树的操作

1、 建立一棵线段树

显然，线段树是递归定义的

对于一个线段树中的区间，若 $L < R$ ，则必能被分作 $[L, Mid]$, $[Mid+1, R]$, $Mid = (L+R) \text{ shr } 1$ 。

若 $L=R$ ，则为递归的边界，此时该区间为一个点。

由此思想，建树时需要利用递归（或者模拟堆栈）。

另外，由于这类线段树是二叉树式存储：

节点 T 的左儿子节点是 $T \text{ shl } 1$ ，代表 $[L, Mid]$ 区间。

节点 T 的右儿子节点是 $T \text{ shl } 1+1$ ，代表 $[Mid+1, R]$ 区间。

这样便建立完了一棵线段树。

伪代码：

```
Void Build(int T,L,R)
{
    If (L==R)
    {
        Add values to the leave node;
        Return;
    }

    Int mid=(L+R)>>1;

    Build(T<<1,L,Mid);
    Build(T<<1+1,Mid+1,R);

    Update the current node with his left son and right son;
}
```

其中 `add value` 与 `update root` 都需要在具体的情境中采用具体的过程。

简单举例，在维护区间 `Max` 值时

当 $L=R$ ，因为只有一个数，说明 $[L,R]$ 的最大值便是序列中的原数。

`Tree[T]=List[L](List[R]亦可)`

易知，父区间的最大值便为左右儿子区间最大值的最大值（。。。）

`Tree[T]=Max(Tree[T<<1],Tree[T<<1+1]);`

执行了 `Build(1,1,Len)` 之后，

便建立完了一棵以 $[1,Len]$ 为根区间，节点 1 为根节点的线段树。

至此，建立部分完毕。

2、查询某个区间

线段树最基本的用途便是保存各个区间内的特定值，
但只有保存，并不够，正如程序只有 Input 而没有 Output。
所以线段树是被用来处理某些 RMQ 问题的常用数据结构。

显然，在 $[L,R]$ 中查询 $[LL,RR]$ 的时候有三种情况。

第一种， $[LL,RR]$ 被 $[L, \text{Mid}]$ 包含，此时直接在 $[L, \text{Mid}]$ 中查询 $[LL,RR]$

第二种， $[LL,RR]$ 被 $[\text{Mid}+1, R]$ 包含，此时直接在 $[\text{Mid}+1, R]$ 中查询 $[LL,RR]$ （同理）

第三种，较复杂的一种， $[LL,RR]$ 被 Mid 从中间截断，此时在 $[L, \text{Mid}]$ 中查询 $[LL, \text{Mid}]$ ，在 $[\text{Mid}+1, R]$ 中查询 $[\text{Mid}+1, RR]$ ，然后将两个查询得到的结果进行 Update，返回 Update 的结果。

这样便处理完了查询。

伪代码：

```
Int Query(int T,L,R,LL,RR) //如果需要返回多个结果，可以考虑用 Struct 存储并返回 Struct
{
    If (LL=L&&RR=R)
        Return Tree[t];

    Int Mid=(L+R)>>1
    If (RR<=Mid)
        Return Query(T<<1,L,Mid,LL,RR);
    If (LL>Mid)
        Return Query(T<<1+1,Mid+1,R,LL,RR);

    Int Left=Query(T<<1,L,Mid,LL,Mid);
    Int Right=Query(T<<1+1,Mid+1,R,Mid+1,RR);
    Int Temp;
    Update(Temp,Left,right);
    Return Temp;
}
```

再以查询区间最大值为例

当 $LL=L$ 且 $RR=R$

返回的即为区间 $[L,R]$ 的最大值，

当全部落在 $[L, \text{Mid}]$ ，便在区间 $[L, \text{Mid}]$ 内查询 $[LL,RR]$ ，

当全部落在 $[\text{Mid}+1, R]$ ，便在区间 $[\text{Mid}+1, R]$ 查询 $[LL,RR]$ 。

当被 Mid 截断，便查询 $[L, \text{Mid}]$, $[\text{Mid}+1, R]$ ，并返回这两个区间查询结果 Update 后的结果。

执行完 $\text{Ans}=\text{Query}(1,1,N,LL,RR)$ ，

便得到了在 $[1,N]$ 查询 $[LL,RR]$ 的结果

至此，查询部分完毕。

3、修改某个区间或点

当某个问题需要查询并修改一个区间或者点的值的时候，线段树也能支持这一操作。

本质上，建立一棵线段树相当于向一棵线段树加入许多点，或者说将原来空的点修改为值。线段树支持修改，第一种是将某个点的值改变，并递归更新他的父亲，第二种是将某个区间加上一个 δ 值，或者改变为一个 $cover$ 值。

修改点：

显然，在 $[L,R]$ 中修改 $[LL,RR]$ ($LL=RR$) 的时候有三种情况。

第一种， $[LL,RR]$ 被 $[L,Mid]$ 包含，此时直接在 $[L,Mid]$ 中修改 $[LL,RR]$

第二种， $[LL,RR]$ 被 $[Mid+1,R]$ 包含，此时直接在 $[Mid+1,R]$ 中修改 $[LL,RR]$ （同理）

然后递归更新父亲。

不用考虑被截断（因为只是一个点）。

修改区间

显然，在 $[L,R]$ 中修改 $[LL,RR]$ ($LL<RR$) 的时候有三种情况。

第一种， $[LL,RR]$ 被 $[L,Mid]$ 包含，此时直接在 $[L,Mid]$ 中修改 $[LL,RR]$

第二种， $[LL,RR]$ 被 $[Mid+1,R]$ 包含，此时直接在 $[Mid+1,R]$ 中修改 $[LL,RR]$ （同理）

第三种，较复杂的一种， $[LL,RR]$ 被 Mid 从中间截断，此时在 $[L,Mid]$ 中修改 $[LL,Mid]$ ，在 $[Mid+1,R]$ 中修改 $[Mid+1,RR]$ ，然后将两个查询得到的结果进行 $Update$ ，递归更新父亲。

这样便处理完了修改。

伪代码（修改区间的，修改点的只需要简单调整）：

```
Void Modify(int T,L,R,LL,RR,Value)
{
    If (LL=L&&RR=R)
        Modify the node with Value

    Int Mid=(L+R)>>1
    If (RR<=Mid)
        Modify(T<<1,L,Mid,LL,RR,Value);
    else
        If (LL>Mid)
            Modify(T<<1+1,Mid+1,R,LL,RR,Value);
        Else
        {
            Modify(T<<1,L,Mid,LL,Mid);
            Modify(T<<1+1,Mid+1,R,Mid+1,RR);
        }
    Update the current node with his left son and right son;
}
```

三、 线段树的应用

解决一些区间 RMQ 问题，存储值，优化 DP 等

习题：

Pku ([Acm.pku.edu.cn/JudgeOnline](http://acm.pku.edu.cn/JudgeOnline))

2104 （较难，树套表，二分答案）

2299 （逆序对）

2777 （区间 Cover）

2823 （最大最小）

3264 （最大最小）

3368 （左右两边连续的长度）

3468 （区间和，支持 delta）